alias_method_chain also has a few more features, such as yielding to a block so that the caller can override the default naming, and dealing with methods that end with an exclamation or a question mark—but essentially, it just builds an *Around Alias* (*134*). Let's see how this mechanism was used in ActiveRecord::Validations.

One Last Look at Validations

Here is the code from the old version of ActiveRecord::Validations again:

```
def self.included(base)
  base.extend ClassMethods
  # ...
  base.class_eval do
    alias_method_chain :save, :validation
    alias_method_chain :save!, :validation
  end
  # ...
end
```

These lines reopen the ActiveRecord::Base class and hack its save and save! methods to add validation. This aliasing ensures that you will get automatic validation whenever you save an object to the database. If you want to save without validating, you can call the aliased versions of the original method, now called save_without_validation.

For the entire scheme to work, the Validations module still needs to define two methods named save_with_validation and save_with_validation!:

gems/activerecord-2.3.2/lib/active_record/validations.rb

```
module ActiveRecord
  module Validations
    def save_with_validation(perform_validation = true)
      if perform_validation && valid? || !perform_validation
        save_without_validation
      else
        false
      end
    end
    def save_with_validation!
      if valid?
        save_without_validation!
      else
        raise RecordInvalid.new(self)
      end
    end
    def valid?
      # ...
```